

Hibernate

Update, Remove and @Embeddable

PART 1 – UPDATE

In Hibernate, we already learned:

```
Insert
└─ persist() → save() [older version]

Select
└─ get()
└─ getReference() → load() [older version]
```

Now we will learn how to **update** an existing object.

Requirement

Suppose a Student with ID **202** already exists in the database.

We want to change the student's degree:

```
Before
-----
ID = 202
Degree = B.tech

After
-----
ID = 202
Degree = M.tech
```

Update Flow

```
Get the existing object
    ↓
Modify the same object
    ↓
Commit the transaction
    ↓
Database gets updated
```

Update Program

```

package com.nit.day05;

import org.hibernate.Session;
import org.hibernate.Transaction;

import com.nit.connectionProvider.ConnectionEstablisher;
import com.nit.entity.Student;

public class UpdateValue {

    public static void main(String[] args) {

        Session currentSession =
            ConnectionEstablisher
                .getConnection()
                .openSession();

        Transaction transcation =
            currentSession.beginTransaction();

        Student reference =
            currentSession.getReference(Student.class, 202);

        System.out.println("Before modification");
        System.out.println(reference);

        reference.setDegree("M.tech");

        System.out.println("After modification");
        System.out.println(reference);

        transcation.commit();

        currentSession.close();
    }
}

```

Important Points

```

Student reference =
    currentSession.getReference(Student.class, 202);

```

Gets a reference to the Student whose ID is **202**.

```

reference.setDegree("M.tech");

```

Modifies the same Student object.

```
transcation.commit();
```

Commits the transaction and synchronizes the change with the database.

Remember

```
graph TD
    A[UPDATE] --> B[Get object]
    B --> C[Modify object]
    C --> D[Commit]
```

PART 2 – REMOVE

Now we will learn how to **remove an existing object**.

Requirement

Suppose a Student with ID **202** exists in the database.

We want to delete that Student.

Remove Flow

```
graph TD
    A[Get the existing object] --> B[remove()]
    B --> C[Commit the transaction]
    C --> D[Data removed]
```

Remove Program

```
import org.hibernate.Session;
import org.hibernate.Transaction;

import com.nit.connectionProvider.ConnectionEstablisher;
import com.nit.entity.Student;

public class RemoveValue {

    public static void main(String[] args) {

        Session currentSession =
```

```

        ConnectionEstablisher
            .getConnection()
            .openSession();

    Transaction transcation =
        currentSession.beginTransaction();

    Student reference =
        currentSession.getReference(Student.class, 202);

    currentSession.remove(reference);

    System.out.println("Data removed");

    transcation.commit();

    currentSession.close();
}
}

```

Important Points

```

Student reference =
    currentSession.getReference(Student.class, 202);

```

Gets the Student object whose ID is **202**.

```

currentSession.remove(reference);

```

Tells Hibernate to remove that object.

```

transcation.commit();

```

Commits the transaction.

Remember

```

REMOVE
  ↓
Get object
  ↓
remove()
  ↓
Commit

```

UPDATE vs REMOVE

```
UPDATE
-----
Get object
  ↓
Modify object
  ↓
Commit
  ↓
Data updated
```

```
REMOVE
-----
Get object
  ↓
remove()
  ↓
Commit
  ↓
Data removed
```

PART 3 – @EMBEDDABLE

Requirement

Suppose we have a Student:

```
Student
-----
id
name
degree
address
```

The Student also has Address information:

```
Address
-----
pinCode
street
landMark
```

The Address information belongs to the Student.

We want to use the **Address** class as a part of the **Student** object.

For this requirement, we use:

Definition of @Embeddable

@Embeddable is a JPA annotation used to mark a class as an **embeddable class**.

In simple words:

@Embeddable
↓
Marks a class as a class
whose fields can be used as part
of another entity.

Example:

```
Student
|
└─ Address
    |
    ├── pinCode
    ├── street
    └─ landMark
```

Here, **Address** is not the main entity in our requirement.

It is used as a part of **Student**.

WHERE DO WE USE @EMBEDDABLE?

We use **@Embeddable** when:

1. A class contains related information.
2. That information belongs to another entity.
3. We want to use that class as a part of the entity.
4. We do not want to treat that class as the main entity for this requirement.

Example:

```
Student
|
└─ Address

Employee
```

```
|
└─ Address
```

Address contains related information such as:

```
pinCode
street
landMark
```

So Address can be used as an embeddable class.

WHEN DO WE USE @EMBEDDABLE?

Use **@Embeddable** when you have a group of fields that logically belong together and are used as a part of another entity.

For example:

```
Student
└─ Address
    │
    ├── pinCode
    ├── street
    └─ landMark
```

Here, Address is a group of related fields.

So we can make **Address** an **@Embeddable** class.

Simple Rule

If the class represents a main entity

↓

Use @Entity

If the class represents a group of fields
that will be used as a part of another entity

↓

Use @Embeddable

Address Class

```
package com.nit.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;
```

```
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.Data;

@Entity
@Data
@Embeddable
public class Address {

    @Column(name="area_pincode")
    private String pinCode;

    private String street;

    @Column(name="near_by")
    private String landMark;
}
```

Important Part

```
@Embeddable
public class Address
```

This marks **Address** as an embeddable class.

The Address contains:

```
pinCode
street
landMark
```

These fields represent Address information.

Student Class

```
package com.nit.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.Data;
```

```

@Entity
@Table(name="student_values")
@Data
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int id;

    @Column(name="student_name")
    private String name;

    @Column(name="student_degree")
    private String degree;

    @Column(name="student_address")
    private Address address;

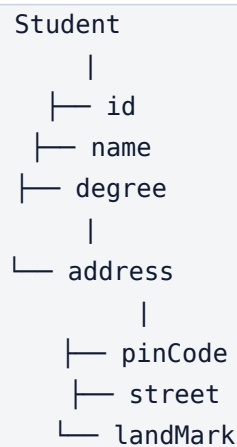
    // pin_code , street , landMark
}

```

The Student contains an Address object:

```
private Address address;
```

So the structure is:



ADD ADDRESS TO STUDENT

First, create the Address object:

```
Address address = new Address();
```

Set its values:

```
address.setPinCode("523523");
address.setStreet("H.tech");
address.setLandMark("near by oracle");
```

Now create the Student object:

```
Student student1 = new Student();

student1.setName("abc");
student1.setDegree("B.tech");
student1.setAddress(address);
```

Here:

```
student1.setAddress(address);
```

sets the Address object inside the Student object.

AddAddress Program

```
package com.nit.day05;

import org.hibernate.Session;
import org.hibernate.Transaction;

import com.nit.connectionProvider.ConnectionEstablisher;
import com.nit.entity.Address;
import com.nit.entity.Student;

public class AddAddress {

    public static void main(String[] args) {

        Session currentSession =
            ConnectionEstablisher
                .getConnection()
                .openSession();

        Transaction transcations =
            currentSession.beginTransaction();

        Address address = new Address();

        address.setPinCode("523523");
        address.setStreet("H.tech");
        address.setLandMark("near by oracle");
```

```

Student student1 = new Student();

student1.setName("abc");
student1.setDegree("B.tech");
student1.setAddress(address);

currentSession.persist(student1);

System.out.println("Data added!!");

System.out.println("Fetching the data");

Student reference =
    currentSession
        .getReference(Student.class, 252);

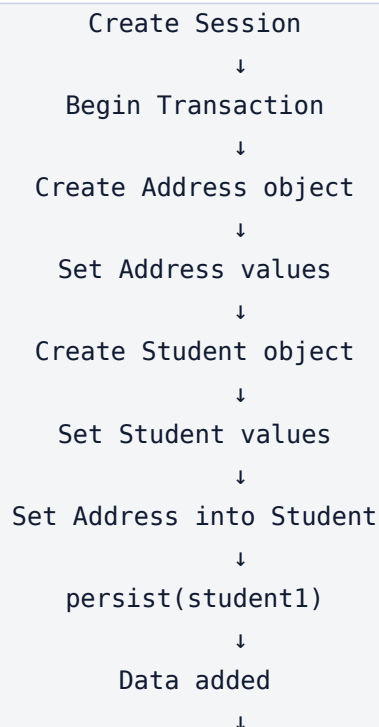
System.out.println(reference);

transcations.commit();

currentSession.close();
}
}

```

AddAddress Program Flow



```
getReference()
  ↓
Display Student
  ↓
commit()
  ↓
Close Session
```

KEY POINTS

`persist()`
→ Used to insert the object.

`getReference()`
→ Used to get a reference to an existing object.

UPDATE
→ Get the object
→ Modify the object
→ Commit

`remove()`
→ Removes an existing object.

`@Entity`
→ Used for the main entity class.

`@Embeddable`
→ Marks a class whose fields can be used
as a part of another entity.

@ENTITY vs @EMBEDDABLE

```
@Entity
  ↓
Main entity
  ↓
Represents the main object
```

@Embeddable
↓
Part of an entity
↓
Contains a group of related fields

Simple Example

```
@Entity
Student
|
|— id
|— name
|— degree
|
|— Address
    |
    |— pinCode
    |— street
    |— landMark
```

Here:

```
Student
  → Main entity

Address
  → Embeddable class
```

MEMORY TRICK

@Entity
↓
"This is the main entity."

@Embeddable
↓
"This class is a part of another entity."

UPDATE
↓

Get → Modify → Commit

REMOVE

↓

Get → remove() → Commit